QuillAudits

AUDIT REPORT

May 2026

For

elsa

Table of Content

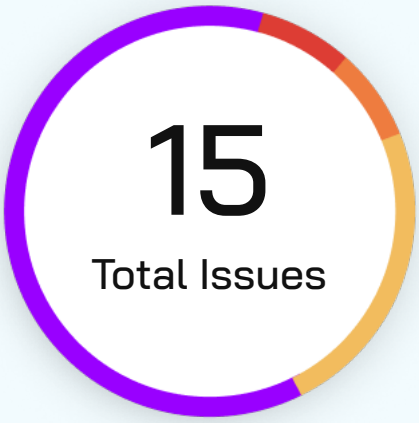
Executive Summary

Project Name	HeyElsa
Protocol Type	Staking
Project URL	https://www.heyelsa.ai/
Overview	HeyElsa Staking protocol implements a vote-escrow staking mechanism where users lock ELSA tokens to receive veELSA, which represents time-weighted voting power. veELSA decays linearly over time and is used for: • Season 1 reward distribution (TVL-scaled proportionate emissions) • Governance voting power • Ecosystem airdrop eligibility
Audit Scope	The scope of this Audit was to analyze the HeyElsa Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/HeyElsa/staking-contract-v2
Branch	Main
Contracts in Scope	VotingEscrow.sol, RewardDistributor.sol
Commit Hash	39524073dfed17ef6ac6d013bd5b5137fae814a0
Language	Solidity
Blockchain	EVM
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	28th April 2026 - 7th May 2026
Updated Code Received	8th May 2026
Review 2	9th May 2026
Fixed In	97b55753bc3597ab60d2fb72012cb1ff4534e26e

Verify the Authenticity of Report on QuillAudits Leaderboard:

https://www.quillaudits.com/leaderboard

Number of Issues per Severity



Critical	0 (0.00%)
High	1 (6.5%)
Medium	1 (6.5%)
Low	4 (27.0%)
Informational	9 (60.0%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	0	0	**3**
Partially Resolved	0	0	0	0	0
Resolved	0	**1**	**1**	**4**	**6**

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Asymmetric eligibility-window guard inflates the eligible-ve denominator and permanently dilutes honest staker's rewards	High	Resolved
2	Reward Dust Locked Due to Integer Division	Medium	Resolved
3	RewardsClaimed Event Emits Input `toEpoch` Instead of Actual Last-Processed Epoch	Low	Resolved
4	`_withdraw` Silent No-Op When No Lock Exists Obscures Caller Errors	Low	Resolved
5	No Rescue Function for Accidentally Sent ERC-20 or ETH Tokens	Low	Resolved
6	rewardsMax Cap Not Applied in Reward View Functions, Overstating Available Rewards	Low	Resolved
7	Multiple Admin State-Change Functions Emit No Events	Informational	Resolved
8	increaseUnlockTime Emits No Event When Lock Extended	Informational	Acknowledged
9	SeasonEnded Event Missing epochsCheckpointed or Rewards Distributed Field	Informational	Resolved

Issue No.	Issue Title	Severity	Status
10	wasSeasonEligible Flag Cannot Be Cleared After Season Transition	Informational	Resolved
11	Ownership Transfer Uses Single-Step Ownable Instead of Ownable2Step	Informational	Resolved
12	endSeason Blocked if Any Epoch in Range Not Checkpointed	Informational	Acknowledged
13	claimPartial cap Parameter Can Exceed Total Epochs, Silently Processing All	Informational	Resolved
14	MULTIPLIER and PRECISION Constants Use Non-Standard Naming Convention	Informational	Acknowledged
15	totalClaimable View Function Has Unbounded Loop, Potential Gas Exhaustion for On-Chain Callers	Informational	Resolved

Checked Vulnerabilities

- Access Management
- Arbitrary write to storage
- Centralization of control
- Ether theft
- Improper or missing events
- Logical issues and flaws
- Arithmetic Computations Correctness
- Race conditions/front running
- SWC Registry
- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- Exception Disorder
- Gasless Send
- Use of tx.origin
- Malicious libraries
- Compiler version not fixed
- Address hardcoded
- Divide before multiply
- Integer overflow/underflow
- ERC's conformance
- Dangerous strict equalities
- Tautology or contradiction
- Return values of low-level calls

- Missing Zero Address Validation
- Private modifier
- Revert/require functions
- Multiple Sends
- Using suicide
- Using delegatecall
- Upgradeable safety
- Using throw
- Using inline assembly
- Style guide violation
- Unsafe type inference
- Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

High Severity Issues

Asymmetric eligibility-window guard inflates the eligible denominator and permanently dilutes honest staker's rewards

Resolved

Path

VotingEscrow.sol

Function

`_increaseUnlockTime()`

Description

VotingEscrow exposes three sister entry points for changing a lock: _depositFor (used by depositFor), _increaseAmount (used by increaseAmount), and _increaseUnlockTime (used by increaseUnlockTime and increaseAmountAndTime). Two of those three reject any mutation by a seasonEligible user once the eligibility window has closed the third does not.

_increaseAmount and depositFor both contain :

```
if (seasonEligible[msg.sender] && eligibilityEnd != 0 && block.timestamp > eligibilityEnd) {
    revert EligibilityWindowClosed();
}
```

_increaseUnlockTime checks only lock.end, lock.amount, and the rounded-time bounds it has no eligibility-window guard:

```
function _increaseUnlockTime(uint256 unlockTime) internal {
    LockedBalance memory lock = _locked[msg.sender];
    uint256 roundedUnlockTime = (unlockTime / WEEK) * WEEK;

    if (lock.end <= block.timestamp) revert LockExpired();
    if (lock.amount == 0) revert NoLockFound();
    if (roundedUnlockTime <= lock.end) revert CanOnlyIncreaseLockDuration();
    if (roundedUnlockTime > block.timestamp + MAXTIME) {
        revert UnlockTimeTooLong();
    }

    _depositFor(msg.sender, 0, roundedUnlockTime, lock, DepositType.INCREASE_UNLOCK_TIME);
}
```

Because _depositFor calls _checkpointEligible for any seasonEligible user, an eligible locker can extend their unlock time long after eligibilityEnd has passed. The downstream effect is an inflation of the eligible-ve curve: _eligiblePointHistory records a new point whose bias is computed from the extended end, and _eligibleSlopeChanges[newEnd] schedules slope removal at that future date. Every subsequent call to eligibleVeAtT(t) for t >= extension_timestamp returns an inflated value.

This corrupts the reward accounting in RewardDistributor.checkpointEpoch. On the first read for an affected epoch the contract executes:

```
uint256 totalVe = votingEscrow.eligibleVeAtT(epochStartTime);
...
data.totalEligibleVe = totalVe;
data.checkpointed = true;
```

data.totalEligibleVe is then frozen re-checkpoint reverts with EpochAlreadyCheckpointed . There is no admin reset, no overwrite, no re-checkpoint method anywhere in the codebase. Every honest user's epoch share userVe / data.totalEligibleVe is now divided by an inflated denominator while their numerator (computed from their unchanged user-point history) is unchanged. The deficit suffered by every honest user is mathematically transferred to the user who extended their lock.

The asymmetry is also reachable via increaseAmountAndTime, which routes the time-extension portion through the same unguarded _increaseUnlockTime internal path.

Impact

Permanent, on-chain, deterministic loss of reward accrual for every honest eligible user for every epoch checkpointed at or after the post-window extension.
Loss is unrecoverable without contract redeployment _epochData[epochId] is immutable once checkpointed = true.

Likelihood

High

POC

```
    function test_increaseUnlockTime_succeeds_after_window_close() public {
        // Lock during eligibility window.
        vm.warp(rewardDistributor.seasonStart() - 1 days);
        createLock(bob, 30_000_000 ether, 2 * YEAR);
        assertTrue(votingEscrow.seasonEligible(bob), "bob is eligible");

        // Move past eligibilityEnd.
        vm.warp(rewardDistributor.eligibilityEnd() + 1 days);

        // SISTER PATH: increaseAmount() reverts -> guard works there.
        vm.prank(bob, bob);
        vm.expectRevert(VotingEscrow.EligibilityWindowClosed.selector);
        votingEscrow.increaseAmount(1_000 ether);

        // BUG: increaseUnlockTime() succeeds -> guard MISSING here.
        // Pick a new unlock time roughly = now + MAXTIME (rounded down to a
week).
        uint256 newUnlock = roundToWeek(block.timestamp + MAXTIME) - WEEK; //
safe <= now+MAXTIME
        vm.prank(bob, bob);
        votingEscrow.increaseUnlockTime(newUnlock);

        // Confirm the lock end actually moved -> the post-window write
happened.
        ( , uint256 lockEnd) = votingEscrow.locked(bob);
        assertEq(lockEnd, newUnlock, "lock.end was extended after eligibility
window");
    }
```

Recommendation

Add the three-line guard mirroring the exact pattern used in _increaseAmount and depositFor

Medium Severity Issues

Reward Dust Locked Due to Integer Division

Resolved

Path

RewardDistributor.sol

Function Name

`getRewardsForEpoch()`

Description

getRewardsForEpoch divides rewardsMax by totalEpochs using integer arithmetic:

```
function getRewardsForEpoch(uint256) public view returns (uint256) {
    uint256 totalEpochs = (seasonEnd - seasonStart) / epochLength;
    if (totalEpochs == 0) return 0;
    return rewardsMax / totalEpochs;
}
```

When rewardsMax is not evenly divisible by totalEpochs, the remainder is silently discarded. Across the full season, each epoch distributes floor(rewardsMax / totalEpochs) tokens, and the difference rewardsMax - (perEpoch × totalEpochs) — the dust — remains locked in the contract.

For the deployed parameters (rewardsMax = 30,000,000 ELSA, totalEpochs = 43):
perEpoch = 30,000,000 / 43 = 697,674 ELSA
dust = 30,000,000 - (697,674 × 43) = 18 ELSA
The dust amount is small in this configuration, but the issue scales with rewardsMax and the number of epochs chosen in future seasons.

Impact

Reward Dust Locked Due to Integer Division

Likelihood

High

POC

```
function testRewardDustStuckInContract() public {
    uint256 totalEpochs = (rewardDistributor.seasonEnd() -
rewardDistributor.seasonStart()) / WEEK;
    uint256 perEpoch = rewardDistributor.getRewardsForEpoch(0);
    uint256 dust = rewardDistributor.rewardsMax() - (perEpoch * totalEpochs);

    // dust > 0 when rewardsMax is not divisible by totalEpochs
    // no function distributes this remainder — it stays in the contract
forever
}
```

Recommendation

Distribute the dust in the final epoch by detecting and adding the remainder:

```
function getRewardsForEpoch(uint256 epochId) public view returns (uint256) {
    uint256 totalEpochs = (seasonEnd - seasonStart) / epochLength;
    if (totalEpochs == 0) return 0;
    uint256 perEpoch = rewardsMax / totalEpochs;
    // add remainder to the last epoch so nothing is stranded
    if (epochId == totalEpochs - 1) {
        perEpoch += rewardsMax % totalEpochs;
    }
    return perEpoch;
}
```

Low Severity Issues

RewardsClaimed Event Emits Input `toEpoch` Instead of Actual Last-Processed Epoch

Resolved

Path

RewardDistributor.sol

Description

The RewardsClaimed event at RewardDistributor.sol is emitted with the caller-supplied toEpoch argument rather than the actual lastProcessedEpoch value computed during claim execution. If the claim loop terminates early (for example because an epoch is not yet checkpointed, has zero eligible supply, or the caller requested a range exceeding available data), the emitted toEpoch will be higher than the epoch through which rewards were actually distributed. Off-chain indexers and analytics systems will incorrectly believe the user was processed through the full requested range.

Impact

Off-chain reward dashboards and analytics display incorrect claim history. Indexers that mark epochs as claimed based on the event data may skip re-processing epochs that were not actually processed. No direct fund loss; the on-chain state (lastProcessedEpoch[user]) is written correctly.

Recommendation

Change the RewardsClaimed emission to use the actual lastProcessedEpoch[user] value (the cursor position after the loop completes) instead of the input toEpoch.

_withdraw Silent No-Op When No Lock Exists Obscures Caller Errors

Resolved

Path

VotingEscrow.sol

Description

The _withdraw internal function at VotingEscrow.sol checks whether the user holds an active lock by comparing lock.end against zero. However, because lock.end is initialised to zero for addresses with no lock, and a withdrawn lock also has lock.end set to zero, the guard condition cannot distinguish between a user who never had a lock and a user whose lock has already been withdrawn. In both cases, the function completes silently without reverting, causing callers to believe the withdrawal succeeded even when no state was changed.

Impact

Callers and integrators cannot distinguish between a successful withdrawal and a no-op caused by a missing or already-withdrawn lock. This can mask user errors (double-withdrawal attempts) and makes auditing more difficult since the transaction succeeds with no indication of what happened.

Recommendation

Use a distinct sentinel value (for example, uint256(1)) for a lock that has been withdrawn, so that lock.end == 0 unambiguously means no lock ever created. Alternatively, revert with a descriptive error when _withdraw is called with no active lock.

No Rescue Function for Accidentally Sent ERC-20 or ETH Tokens

Resolved

Path

VotingEscrow.sol

Description

VotingEscrow.sol contains no administrative function that allows the owner to recover ERC-20 tokens or ETH accidentally sent directly to the contract address. Because the contract holds ELSA tokens for locked positions, any token accidentally sent to the contract (wrong token, user error, airdrop to the staking address) will be permanently locked.

Impact

Tokens accidentally transferred to VotingEscrow.sol are permanently lost with no recovery path. This applies to ERC-20 tokens other than ELSA and to any ETH sent to the contract (which has no receive() or fallback()).

Recommendation

Add a rescueTokens(address token, address recipient, uint256 amount) function restricted to the owner, with a guard preventing withdrawal of the ELSA staking token. Similarly, consider adding a rescueETH function or a receive() handler that immediately forwards ETH to the owner.

rewardsMax Cap Not Applied in Reward View Functions, Overstating Available Rewards

Resolved

Path

RewardDistributor.sol

Description

The view functions getClaimableRewards and getTotalClaimableRewards at RewardDistributor.sol compute a user reward amount without applying the rewardsMax cap. The actual claim path applies the cap, but the view functions return an uncapped (potentially higher) value. Users who consult these views to decide whether to claim will see a larger amount than they will actually receive.

Impact

Users are shown inflated claimable reward estimates, leading to confusion and failed expectations when the actual claimed amount is lower. Front-ends that display rewards based on these view functions will display incorrect data. No direct fund loss.

Recommendation

Apply the same rewardsMax capping logic in the view functions that is applied in the actual claim path. If the cap is not applied in views for performance reasons, add a comment clearly stating that the returned value may exceed the actual claimable amount.

Informational Issues

Multiple Admin State-Change Functions Emit No Events

Resolved

Path

VotingEscrow.sol, RewardDistributor.sol

Description

Several administrative state-change functions across both contracts do not emit events when they modify critical protocol parameters. Functions such as setEligibilityEnd, setRewardsMax, setClaimStart, and others modify values that directly affect user rewards and eligibility, but make no on-chain announcement of the change. Off-chain monitoring systems and indexers have no way to detect these parameter changes without polling storage slots.

Recommendation

Add events for all state-changing admin functions: EligibilityEndUpdated(uint256 oldEnd, uint256 newEnd), RewardsMaxUpdated(uint256 oldMax, uint256 newMax), and so on. Emit these events with`both old and new values to enable monitoring diffs.

increaseUnlockTime Emits No Event When Lock Extended

Acknowledged

Path
VotingEscrow.sol

Function Name
`increaseUnlockTime()`

Description
The increaseUnlockTime function modifies a user lock end time and updates the ve-checkpoint history, but emits no event recording that the lock was extended. The analogous createLock and increaseAmount functions emit events; increaseUnlockTime does not. Off-chain systems tracking user lock durations cannot detect lock extensions without reading storage.

Recommendation
Emit an event from increaseUnlockTime: LockDurationIncreased(address indexed provider, uint256 newUnlockTime, uint256 timestamp).

HeyElsa Team's Comment
Emit an event from increaseUnlockTime: LockDurationIncreased(address indexed provider, uint256 newUnlockTime, uint256 timestamp).

SeasonEnded Event Missing epochsCheckpointed or Rewards Distributed Field

Resolved

Path

RewardDistributor.sol

Function Name

`endSeason()`

Description

The SeasonEnded event emitted by endSeason does not include fields indicating how many epochs were checkpointed or the total rewards distributed during the season. Consumers of this event (governance dashboards, analytics, season-transition scripts) receive minimal information about the completed season and must perform additional storage reads to reconstruct this data.

Recommendation

Add fields to the SeasonEnded event: SeasonEnded(uint256 indexed seasonId, uint256 epochsCheckpointed, uint256 totalRewardsDistributed, uint256 timestamp).

wasSeasonEligible Flag Cannot Be Cleared After Season Transition

Resolved

Path

VotingEscrow.soll

Function Name

`wasSeasonEligible()`

Description

The wasSeasonEligible[user] flag is set to true when a user crosses the eligibility threshold, but there is no mechanism to clear it if eligibilityEnd is subsequently updated or a new season is started. A user who was eligible in season 1 will carry a true flag into season 2, even if their lock has since expired or they no longer meet the eligibility criteria.

Recommendation

Document clearly that wasSeasonEligible is a per-season flag and must be reset at the start of each new season. Consider adding a clearEligibilityFlags administrative function or incorporating a season-id into the mapping key: wasSeasonEligible[seasonId][user].

Ownership Transfer Uses Single-Step Ownable Instead of Ownable2Step

Resolved

Path

VotingEscrow.sol, RewardDistributor.sol

Description

Both VotingEscrow.sol and RewardDistributor.sol inherit from OpenZeppelin Ownable, which implements single-step ownership transfer. A call to transferOwnership(newOwner) immediately transfers control without requiring the new owner to accept. If an incorrect address is supplied (a zero address, wrong EOA, or compromised contract), ownership is transferred irreversibly in a single transaction.

Recommendation

Replace Ownable with Ownable2Step (available in OpenZeppelin v4.9 and later). Ownable2Step requires the new owner to call acceptOwnership() to complete the transfer.

endSeason Blocked if Any Epoch in Range Not Checkpointed

Acknowledged

Path

RewardDistributor.sol

Function Name

`endSeason()`

Description

The endSeason function at RewardDistributor.sol requires all epochs from 0 to the final epoch to be checkpointed before the season can be concluded. If any single epoch in the range is missing its checkpoint, the completeness check will fail and endSeason will revert. A single missed epoch blocks the entire season conclusion indefinitely.

Recommendation

Consider adding a batch checkpoint function that can process multiple epochs in a single transaction. Alternatively, implement an administrative override that allows the owner to finalize a season with a minimum fraction of epochs checkpointed, with the remaining reward fraction refunded or carried forward.

HeyElsa Team's Comment

The strict completeness check is by design, allowing partial season finalization would break reward accounting integrity.

claimPartial cap Parameter Can Exceed Total Epochs, Silently Processing All

Resolved

Path

RewardDistributor.sol

Function Name

`claimPartial()`

Description

The claimPartial function at RewardDistributor.sol accepts a cap parameter limiting how many epochs are processed per call. If cap is larger than the number of remaining claimable epochs, the function silently processes all remaining epochs rather than reverting or warning. Callers who use cap as a strict gas limit rather than an upper bound may process more epochs than expected.

Recommendation

Document in NatSpec that the cap parameter is an upper bound on epochs processed per call, not a strict limit. If strict enforcement is desired, revert when cap > (toEpoch - lastProcessedEpoch[user]).

MULTIPLIER and PRECISION Constants Use Non-Standard Naming Convention

Acknowledged

Path

VotingEscrow.sol, RewardDistributor.sol

Description

Several precision and scaling constants in both contracts are named MULTIPLIER or PRECISION without indicating their magnitude (for example, whether they are 1e6, 1e18, or another value). This naming convention makes it harder for readers to quickly verify that arithmetic operations using these constants are correctly scaled, and increases the risk of scale mismatch bugs being overlooked in code review.

Recommendation

Rename constants to include their magnitude: PRECISION_1E18, MULTIPLIER_1E6, etc. Alternatively, add inline comments indicating the unit and magnitude of each constant.

HeyElsa Team's Comment

We changed MULTIPLIER = 1 ether to MULTIPLIER = 1e18 to make the magnitude explicit. We didn't rename to MULTIPLIER_1E18 since these are fork-inherited constants from Curve/Stargate and we want to keep the diff minimal against upstream.

totalClaimable View Function Has Unbounded Loop, Potential Gas Exhaustion for On-Chain Callers

Resolved

Path

RewardDistributor.sol

Function Name

`totalClaimable`

Description

The totalClaimable view function iterates over all unclaimed epochs for a user from lastProcessedEpoch(user) to the current epoch. As the number of unclaimed epochs grows (for inactive users or after a long season), this loop may consume more gas than the block gas limit allows for on-chain callers. While view functions do not consume gas when called off-chain, on-chain integrations (such as keeper scripts or automated claim systems) that call this function will fail for users with many unclaimed epochs.

Recommendation

Add an optional cap parameter to totalClaimable that limits the number of epochs iterated. Alternatively, document the maximum safe number of unclaimed epochs for on-chain callers and encourage regular claiming to avoid accumulation.

Functional Tests

Some of the tests performed are mentioned below:

- createLock records lock and sets eligibility correctly
- claim advances cursor and transfers correct amount
- checkpointEpoch records correct values and is immutable
- TVL-scaled emission at 0%, 25%, 50%, 100% of tvlTarget
- ve voting power decays linearly to zero at lock end
- Access control — onlyOwner and pause modifiers reject unauthorized calls
- withdraw after expiry returns full principal and clears state
- totalRewardsDistributed never exceeds rewardsMax

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

VotingEscrow

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Ownable (OZ)	Base Contract	Single-owner access control gating all admin functions: setEligibilityEnd, freezeEligibilityEnd, addToBlacklist, removeFromBlacklist, unlock	Owner address is a secured multisig or EOA; private key is not compromised	Compromise of owner key grants immediate ability to call unlock(), allowing all users to withdraw regardless of lock expiry; no timelock on owner functions in this contract
SafeERC20 (OZ)	Library	safeTransferFrom used to pull ELSA from users on lock creation and deposits; safeTransfer used to return ELSA on withdrawal	ELSA token conforms to ERC20 standard and does not introduce transfer hooks that re-enter VotingEscrow	Non-standard ERC20 behaviour (fee-on-transfer, rebase, hooks) on the ELSA token would corrupt supply accounting

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
IRewardDistributor	External Contract	freezeEligibilityEnd(address rewardDistributor_) reads IRewardDistributor(rewardDistributor_).eligibilityEnd() to verify both contracts agree before freezing	The address passed is the legitimately deployed RewardDistributor with correct eligibilityEnd	Owner could pass a malicious contract that returns any eligibilityEnd value, bypassing the mismatch check and permanently freezing an incorrect value; mitigated by the owner trust assumption

Ownable (OZ)

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Ownable (OZ)	Base Contract	Owner gates initialize, pause/unpause variants, endSeason, scheduleEmergencyWithdraw, executeEmergencyWithdraw	Owner is a secured multisig; private key is not compromised	Compromised owner key allows scheduling and executing emergency withdrawals draining the full 30M ELSA reward pool; mitigated by 48h timelock

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Reentrancy Guard (OZ)	Base Contract	nonReentrant on claim, claimFor, claimPartial to prevent double-claim via reentrancy	Mutex correctly prevents re-entry across all claim paths	N/A
Pausable (OZ)	Base Contract	whenNotPaused modifier on checkpointEpoch, checkpointEpochs, claim, claimFor, claimPartial; supplemented by checkpointPaused and claimPaused booleans for granular control	Pause state is correctly checked before any checkpointing or claiming	Owner can pause claiming indefinitely, blocking users from receiving earned rewards; accepted admin risk
IVotingEscrow	External Contract	eligibleVeAtT(epochStart) used as totalEligibleVe denominator per epoch; eligibleSupplyAtT(epochStart) used as eligibleTVL for emission scaling; balanceOfAtT(user, epochStart) used as per-user numerator; wasSeasonEligible(user) gates all claim access; immutable votingEscrow address set in constructor	VotingEscrow correctly implements all four view functions; eligible point history accurately tracks only season-eligible users; wasSeasonEligible is never set for ineligible addresses	Any bug in VotingEscrow's eligibleVeAtT or eligibleSupplyAtT directly corrupts epoch denominators and TVL scaling, leading to incorrect reward distribution; entire reward correctness depends on VotingEscrow's eligible point history accuracy

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

VotingEscrow

Contract Name	Function	Category
VotingEscrow	createLock(uint256 value, uint256 unlockTime)	**What this function can do** Create a new time-locked position for msg.sender; transfers ELSA from msg.sender to the contract; sets seasonEligible = true and wasSeasonEligible = true if block.timestamp ⩽ eligibilityEnd; updates global and eligible-only point histories; increments supply and eligibleSupply if eligible. **What this function cannot/should not do** Be called if a lock already exists for msg.sender (LockExists); lock an amount below MIN_LOCKED_AMOUNT (500 ELSA) (BelowMinimumLockAmount); set unlockTime below MINTIME or above MAXTIME from now; be called by a blacklisted address (ContractBlacklisted); be called when globally unlocked (GloballyUnlocked). **Main invariant(s)** _locked[msg.sender].amount = value (rounded to week); supply += value; if seasonEligible, eligibleSupply += value and wasSeasonEligible = true; lockEnd is week-aligned. Access level External, nonReentrant, onlyNotBlacklisted, notUnlocked

Contract Name	Function	Category
VotingEscrow	depositFor(address addr, uint256 value)	**What this function can do** Allow a third party (msg.sender) to deposit ELSA into addr's existing unexpired lock; useful for protocol-level top-ups; updates supply, eligibleSupply (if addr is seasonEligible), and point histories. **What this function cannot/should not do** Be called with zero value (ZeroValue); deposit into a nonexistent lock (NoLockFound); deposit into an expired lock (LockExpired); deposit for a seasonEligible addr after eligibilityEnd (EligibilityWindowClosed); be called by or for a blacklisted address (ContractBlacklisted checked for both msg.sender and addr); be called when globally unlocked. **Main invariant(s)** addr's locked amount increases by value; ELSA transferred from msg.sender (not addr); supply += value; if seasonEligible[addr], eligibleSupply += value. **Access level** External, nonReentrant, notUnlocked — open to any non-blacklisted caller

Contract Name	Function	Category
VotingEscrow	withdraw()	**What this function can do** Return all locked ELSA to msg.sender when lock has expired or when globally unlocked; clears the lock; decreases supply; if seasonEligible, decreases eligibleSupply, removes slope from eligible point history, and clears seasonEligible (but preserves wasSeasonEligible so accrued rewards remain claimable). **What this function cannot/should not do** Withdraw before lock expiry unless unlocked = true (LockNotExpired); withdraw with no lock. **Main invariant(s)** _locked[msg.sender] = LockedBalance(0,0) after withdrawal; supply decreases by withdrawn amount; if was seasonEligible, seasonEligible = false and eligibleSupply decreases; wasSeasonEligible preserved. **Access level** External, nonReentrant — open to all

Contract Name	Function	Category
VotingEscrow	checkpoint()	**What this function can do** Advance the global point history to the current timestamp without any user-specific changes; fills in any weekly slots that have been skipped since the last checkpoint **What this function cannot/should not do** Be called when globally unlocked (notUnlocked reverts). **Main invariant(s)** Global epoch advances; point history filled for all missed weeks up to block.timestamp; no change to any user's lock or supply. **Access level** External, nonReentrant — open to all
VotingEscrow	unlock()	**What this function can do** Permanently set unlocked = true; this allows all users to call withdraw() regardless of whether their lock has expired; disables checkpoint() and all notUnlocked-gated functions. **What this function cannot/should not do** Be reversed — there is no re-lock function; once called the state is permanent. **Main invariant(s)** unlocked = true permanently; all notUnlocked functions revert; any user can withdraw at any time after this call **Access level** External, onlyOwner. RISK: Irreversible emergency escape hatch; compromised owner key combined with this call allows immediate mass withdrawal of all locked principal

RewardDistributor

Contract Name	Function	Category
RewardDistributor	initialize(uint256 rewardsMax_, uint256 tvlTarget_, uint256 seasonStart_, uint256 seasonEnd_, uint256 eligibilityEnd_, uint256 claimStart_)	**What this function can do** Set all season parameters in a single call; rounds seasonStart, seasonEnd, and eligibilityEnd to the nearest week boundary; sets epochLength = WEEK; emits SeasonStarted with a keccak256 hash of all params for integrity verification. **What this function cannot/should not do** Be called more than once (AlreadyInitialized); set rewardsMax or tvlTarget to zero; set a seasonStart already in the past after rounding; set seasonEnd ⩽ seasonStart after rounding; set eligibilityEnd outside (seasonStart, seasonEnd]; set claimStart < seasonEnd. **Main invariant(s)** initialized = true; all params frozen; seasonStart, seasonEnd, eligibilityEnd are week-aligned; epochLength = WEEK. KNOWN ISSUE (M-01): If rewardsMax % ((seasonEnd - seasonStart) / WEEK) != 0, the remainder is locked via normal distribution. **Access level** External, onlyOwner — callable exactly once

Contract Name	Function	Category
RewardDistributor	claim()	**What this function can do** Claim all available rewards for msg.sender across up to MAX_EPOCHS_PER_CLAIM (50) consecutive checkpointed epochs starting from userLastClaimedEpoch; advances the claim cursor; transfers ELSA to msg.sender. **What this function cannot/should not do** Run before claimStart (ClaimingNotEnabled); run for an ineligible user (UserNotEligible via wasSeasonEligible check); run when claimPaused or globally paused; transfer more than the contract holds (InsufficientRewardBalance); exceed rewardsMax (RewardsCapExceeded). **Main invariant(s)** userLastClaimedEpoch[msg.sender] is non-decreasing; totalRewardsDistributed never exceeds rewardsMax; cursor stops at first uncheckpointed epoch and can resume after gap is filled. **Access level** External, nonReentrant, whenNotPaused

Contract Name	Function	Category
RewardDistributor	endSeason()	**What this function can do** Mark the season as concluded; iterates all epochs to verify every one has been checkpointed before allowing the call; emits SeasonEnded with a params hash matching the one from SeasonStarted. **What this function cannot/should not do** Be called before block.timestamp ⩾ seasonEnd (InvalidParameters); be called more than once (SeasonAlreadyEnded); be called when any epoch is uncheckpointed (EpochsNotFullyCheckpointed). **Main invariant(s)** seasonEnded = true; does NOT prevent subsequent claims — users can still collect accrued rewards after season end. **Access level** External, onlyOwner

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
VotingEscrow	ERC20 (ELSA)	All user-locked ELSA principal; held for the duration of each user's lock period (MINTIME to MAXTIME); only releasable on lock expiry, via withdraw(), or via the owner-triggered global unlock()
RewardDistributor	ERC20 (ELSA)	Season 1 reward pool of up to 30,000,000 ELSA; funded once by the protocol treasury at initialization; depleted over time as eligible users claim their proportional epoch rewards; any undistributed amount (due to TVL below target or rounding dust) remains in the contract until emergency withdrawal

3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
VotingEscrow	createLock()	ELSA from msg.sender (safeTransferFrom)	–	Any non-blacklisted address	MIN_LOCKED_AMOUNT (500 ELSA) enforced; lock duration bounded by MINTIME and MAXTIME; eligibility set atomically with deposit
VotingEscrow	increaseAmount()	ELSA from msg.sender (safeTransferFrom)	–	Any non-blacklisted address with an active lock	Blocked for seasonEligible users after eligibilityEnd (EligibilityWindowClosed) — prevents late capital inflation of eligible supply
VotingEscrow	depositFor()	ELSA from msg.sender (safeTransferFrom), credited to addr's lock	–	Any non-blacklisted address	Blocked for eligible recipients after eligibilityEnd; both msg.sender and addr checked against blacklist; token flows from depositor not lock owner
VotingEscrow	increaseAmountAndTime()	ELSA from msg.sender when value > 0	–	Any non-blacklisted address with an active lock	Same eligibility-window restriction as increaseAmount when value > 0; increaseUnlockTime portion is always permitted

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
VotingEscrow	withdraw()	–	Full locked ELSA to msg.sender (safeTransfer)	Any user with an active lock	Requires lock to be expired unless unlocked = true; seasonEligible cleared on withdrawal; wasSeasonEligible preserved for post-withdrawal claiming
RewardDistributor	Treasury transfer (off-chain setup)	Up to 30,000,000 ELSA from protocol treasury via standard ERC20 transfer	–	Protocol treasury / owner	No on-chain enforcement that the full rewardsMax is funded; if underfunded, InsufficientRewardBalance reverts block later claims
RewardDistributor	claim()	–	Proportional ELSA share to msg.sender (safeTransfer)	Any wasSeasonEligible address after claimStart	Capped at rewardsMax total; InsufficientRewardBalance check immediately before transfer; 50-epoch batch limit per call to bound gas
RewardDistributor	claimFor(address user)	–	Proportional ELSA share to user (safeTransfer)	Any address, unless claimForDisabled[user] = true	Same distribution logic as claim(); claimForDisabled opt-out protects users from unwanted third-party claiming

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
RewardDistributor	claimPartial(uint256 fromEpoch, uint256 toEpoch)	–	Proportional ELSA share to msg.sender for specified range (safeTransfer)	Any wasSeasonEligible address after claimStart	fromEpoch silently overridden to userLastClaimedEpoch; toEpoch capped at last claimable epoch; prevents epoch skipping

Closing Summary

In this report, we have considered the security of HeyElsa. We performed our audit according to the procedure described above.

1 High , 1 Medium , 4 Low, 9 Informational issues were found, which have been noted by the team. In the End, the Heyelsa team resolved most of the issues.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With eight years of expertise, we've secured over 1500 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

May 2026

For

elsa

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com